list with solutions for its tail (which is a smaller list). For instance, if you have to develop an algorithm to search the max of a list of numbers, you can:

- If the list is empty, return `null`.
- If the list has at least one element, take the list's head (call it `x`), apply the algorithm to the list with the remaining `N - 1` elements obtaining a value `y`. Then the result is `x` if `y` is `null` or `x >= y; y` otherwise.

C.1.3 Tree

Trees are another widely used abstract data type that provides a hierarchical tree-like structure. They can be seen as a generalization of *linked lists*: each node has still a single predecessor, called its *parent*, but can have more than a single child. Each tree node's child is itself a subtree (either empty, or including a root and its subtrees).

A tree can be formally defined as either:

- An empty tree.
- A node with one or more references to its children, and (optionally) a reference to its parent.

Each tree node, like *list*'s nodes, contains a value. Moreover, a node holds a list of references to other nodes, its *children*. There is a constraint: in a tree each node is the children of just a single other node, except for the root, which has no *parent* (so no node in a *tree* points to its *root*). Therefore, we'll need to keep a reference (using a variable) to the root of the tree. This is the same as a singly linked list where the head of the list is not linked by other nodes. Moreover, trees define a "vertical" hierarchy, a parent-children relation, while there can be no relation between sibling nodes or between nodes in different subtrees.